Method Used by the Submitted secp256k1 Point-Addition Circuit

1 Scope

This document describes the circuit path used by the submission in `src/point_add/` when evaluated by `src/main.rs` through

```
cargo run --release
```

with no environment variables set. Environment-gated diagnostics and alternative experimental paths are intentionally ignored here.

The circuit implements generic affine addition on secp256k1,

$$P = (x_P, y_P), \qquad Q = (x_Q, y_Q), \qquad R = P + Q,$$

where P is quantum and Q is classical. The evaluator avoids point doubling, inverse-pair, and infinity cases, so the implemented path assumes the affine denominators are nonzero.

2 Evaluator Contract

The evaluator in `src/main.rs` calls

```
point_add::build()
```

and then analyzes the emitted operation stream. The required interface is four 256-bit registers, declared in this order:

0 : `target_x`, quantum qubits,
1 : `target_y`, quantum qubits,
2 : `offset_x`, classical bits,
3 : `offset_y`, classical bits.

The input is

$$(\texttt{target\_x}, \texttt{target\_y}) = (x_P, y_P), \qquad (\texttt{offset\_x}, \texttt{offset\_y}) = (x_Q, y_Q),$$

and the output must be

$$(\texttt{target\_x}, \texttt{target\_y}) = (x_R, y_R).$$

The operation stream is hashed with SHAKE256 and the hash-derived stream is used both to generate test points and to seed the simulator. Thus changing the circuit changes the test inputs. The evaluator generates random multiples of the secp256k1 generator, filters out unsupported exceptional cases, runs the circuit in 64-shot batches, and checks:

1. classical correctness against `WeierstrassEllipticCurve::add`;
2. zero global phase after the forward circuit;
3. no non-register qubit garbage after the output registers are zeroed.

The simulator counts `CCX` and `CCZ` as Toffoli gates. It counts `CX`, `CZ`, `Swap`, `Hmr`, and `R` as Clifford gates. `X` and `Z` are free in the reported metrics.

3 Field and Curve Constants

The circuit is specialized to secp256k1:

$$p = 2^{256} - 2^{32} - 977, \qquad E : y^2 = x^3 + 7 \pmod{p}.$$

It uses the Solinas form

$$2^{256} \equiv c \pmod{p}, \qquad c = 2^{32} + 977 = 2^{32} + 2^{10} - 2^6 + 2^4 + 1.$$

This sparse representation is used throughout modular reduction.

4 High-Level Point-Addition Formula

The evaluator's affine addition formula is

$$\lambda = \frac{y_Q - y_P}{x_Q - x_P},$$

$$x_R = \lambda^2 - x_P - x_Q,$$

$$y_R = \lambda(x_P - x_R) - y_P.$$

The circuit first subtracts the classical offset from the quantum target:

$$d_x = x_P - x_Q, \qquad d_y = y_P - y_Q.$$

Since both numerator and denominator are negated,

$$\lambda = \frac{d_y}{d_x}.$$

The implementation stores the negative slope

$$\ell = -\lambda.$$

4.1 First inverse and slope computation

The first Kaliski inversion is run on d_x. For the default no-env path, the iteration count is

$$t_1 = 404.$$

The raw Kaliski output used by the circuit has the form

$$I_1 = -d_x^{-1} 2^{t_1} \pmod{p}.$$

The circuit multiplies

$$\ell_{\text{raw}} = d_y I_1$$

and then applies t_1 modular halvings:

$$\ell = 2^{-t_1}\ell_{\text{raw}} = -\frac{d_y}{d_x} = -\lambda.$$

4.2 Combined affine x- and y-update

Instead of separately zeroing d_y, computing x_R, and then computing y_R, the default path uses a combined affine rearrangement.

Let

$$A = \ell^2 = \lambda^2.$$

The circuit forms a temporary value

$$B = A - 3x_Q.$$

Then it updates the y-register as

$$d_y \leftarrow d_y + \ell B.$$

Using $d_y = \lambda(x_P - x_Q)$ and $\ell = -\lambda$,

$$d_y + \ell(A - 3x_Q) = \lambda(x_P - x_Q) - \lambda(A - 3x_Q) = \lambda(x_P + 2x_Q - A).$$

Since

$$x_R = A - x_P - x_Q,$$

we have

$$y_R + y_Q = \lambda(x_P - x_R) - y_P + y_Q = \lambda(x_P + 2x_Q - A).$$

Thus after the combined update,

$$\texttt{target\_y} = y_R + y_Q.$$

The x-register is updated as

$$d_x \leftarrow -(d_x - A + 3x_Q) = A - d_x - 3x_Q = A - (x_P - x_Q) - 3x_Q = x_R - x_Q.$$

So after the combined affine block,

$$\texttt{target\_x} = x_R - x_Q, \qquad \texttt{target\_y} = y_R + y_Q, \qquad \ell = -\lambda.$$

4.3 Second inverse and slope cleanup

The slope register must be uncomputed. The second Kaliski inversion is run on

$$D = x_R - x_Q.$$

The default iteration count for this second inverse is

$$t_2 = 400.$$

The raw inverse has the form

$$I_2 = -D^{-1}2^{t_2}.$$

Before using it, the circuit doubles the slope register t_2 times:

$$\ell \leftarrow 2^{t_2}\ell.$$

At this point

$$\texttt{target\_y} = y_R + y_Q.$$

From the affine formula,

$$y_R + y_Q = \lambda(x_Q - x_R) = -\lambda(x_R - x_Q) = \ell D.$$

Therefore

$$I_2(y_R + y_Q) = (-D^{-1}2^{t_2})(\ell D) = -2^{t_2}\ell.$$

The circuit adds this product into the slope register, giving zero:

$$2^{t_2}\ell + I_2(y_R + y_Q) = 0.$$

It then subtracts the classical y_Q from the y-register and adds the classical x_Q to the x-register:

$$y_R + y_Q - y_Q = y_R, \qquad x_R - x_Q + x_Q = x_R.$$

The slope register is then clean and can be freed.

5 Kaliski Inversion Subroutine

The default path uses a qrisp-style binary Kaliski almost-inverse circuit. It does not overwrite the input denominator. Instead it allocates a persistent state

$$(u, v, r, s, m_{\text{hist}}, f),$$

where u, v, r, s are 256-bit quantum registers, m_{hist} is an iteration-history register, and f is a one-qubit active/terminal flag.

Initialization is

$$u = p, \qquad v = x, \qquad r = 0, \qquad s = 1, \qquad f = 1.$$

Each iteration computes local branch flags, conditionally swaps (u, v) and (r, s), conditionally performs

$$v \leftarrow v - u, \qquad s \leftarrow s + r,$$

then halves v and doubles r. The swaps are then undone, and the local flags are uncomputed. The branch history is stored in m_{hist}, so the forward computation can later be explicitly reversed.

After the chosen fixed number of iterations, for the denominators exercised by the submitted circuit, the state exposes

$$r = -x^{-1}2^t \pmod{p},$$

with $t = 404$ in the first inverse and $t = 400$ in the second inverse.

The default implementation includes several important optimizations:

1. **Bulk prefix iterations.** Early Kaliski iterations are known, for the tested nonzero denominators, to be nonterminal. The default path uses specialized bulk iterations for this prefix. The no-env caps are 378 bulk iterations for the first inverse and 377 for the second inverse.

2. **Truncated late-iteration widths.** The implementation uses bit-length bounds to shrink comparisons, swaps, and add/subtract slices in late iterations. Roughly, after iteration i, high bits above the bound $2n - i$ are known zero in the denominator state.

3. **Small-r modular doubling shortcut.** For iterations below the threshold 262, the top bit of r is known zero, so modular doubling of r needs no Solinas correction and is emitted as a plain shift.

4. **Measurement-based uncomputation.** ANDs, carry chains, OR chains, and temporary products are often uncomputed by `Hmr` followed by classically-conditioned phase correction. This removes many reverse Toffolis while preserving phase cleanliness.

5. **State lifetime reduction.** After forward Kaliski and before the multiplication body, registers known to be zero or known constants are freed and then reallocated/reloaded for the backward pass. In the default path this reduces the live qubit peak while preserving a clean explicit backward uncomputation.

6 Modular Arithmetic

All arithmetic is over $\mathbb{F}_p$, with $n = 256$.

6.1 Cuccaro adders with measurement-based uncompute

The primitive adder is a Cuccaro ripple-carry adder. The default fast variant computes carries using Toffolis and uncomputes many carry bits using measurement-based reset:

$$\texttt{Hmr}(t, m), \qquad \texttt{CZ}(a, b) \text{ conditioned on } m.$$

This is the standard measurement-based AND-uncompute pattern. The evaluator's global phase check is what enforces that all such measurement feedback is phase-correct.

6.2 Modular addition

For $a, b \in [0, p)$, the circuit computes $a + b \bmod p$ using an extended 257-bit register. Let

$$c = 2^{256} - p = 2^{32} + 977.$$

First the circuit computes the unreduced sum $S = a + b$. It then adds c. The top bit of $S + c$ indicates whether $S \geq p$. If reduction was not needed, the added c is subtracted back. If reduction was needed, the top bit is cleared, yielding $S - p$. The reduction flag is then uncomputed by a comparison identity.

6.3 Modular subtraction

For $a - b \bmod p$, the circuit performs an extended subtraction and records the borrow. If there was an underflow, the wrapped low 256 bits are corrected using the Solinas identity $p \equiv -c \pmod{2^{256}}$. The borrow flag is then uncomputed through a comparison after temporarily negating the subtrahend.

6.4 Doubling and halving

Modular doubling is implemented as a left shift. The old top bit becomes an overflow flag. If the overflow flag is set, the sparse constant c is added.

Modular halving is emitted as the inverse operation: the low bit determines the correction, then a right shift is performed. These operations are used heavily to absorb the 2^t scaling factors produced by Kaliski inversion.

7 Multiplication and Reduction

7.1 Wide product generation

The default path uses reversible wide-product generation followed by Solinas reduction and then Bennett uncomputation of the wide product.

The main schoolbook product primitive is a Litinski-style controlled add-subtract multiplier. For $n = 256$, it uses a $2n + 1$-bit accumulator and performs n controlled add-subtract rows. Corrections are then applied so that the final shifted accumulator contains xy. The wide product is later uncomputed by the exact inverse routine.

7.2 Karatsuba where enabled by default

With no environment variables, Karatsuba is enabled for the main pair-1 write-into-zero multiplication and for the pair-2 cleanup multiplication.

The 1-level Karatsuba split is

$$x = x_0 + 2^{128} x_1, \qquad y = y_0 + 2^{128} y_1,$$

with

$$z_0 = x_0 y_0, \qquad z_2 = x_1 y_1,$$

$$z_1 = (x_0 + x_1)(y_0 + y_1) - z_0 - z_2.$$

The three half-size products are merged into the 512-bit temporary product. After Solinas reduction into the target accumulator, the Karatsuba product is run backward to clean the temporary registers.

7.3 Symmetric squaring

The combined affine block needs ℓ^2. The default path uses a symmetric schoolbook squarer instead of a full multiplier. Cross terms $x_i x_j$ for $i < j$ are generated once and placed with the appropriate factor of two, reducing the Toffoli count of the squaring step.

7.4 Solinas reduction of a 512-bit product

For a wide product

$$T = T_0 + 2^{256} T_1,$$

the Solinas prime gives

$$T \equiv T_0 + cT_1 \pmod{p}.$$

Using

$$c = 2^{32} + 2^{10} - 2^6 + 2^4 + 1,$$

the circuit adds

$$T_0 + T_1 + 2^4 T_1 - 2^6 T_1 + 2^{10} T_1 + 2^{32} T_1$$

modulo p. The high half T_1 is walked through the required shifts by modular doubling and by a specialized 22-bit shift from position 10 to position 32. After the reduction additions are complete, T_1 is shifted/halved back to its original value so that the wide product can be uncomputed.

8 Default No-Environment Circuit Schedule

The no-env path is:

1. Allocate and declare the four interface registers.
2. Compute
$$d_x = x_P - x_Q, \qquad d_y = y_P - y_Q.$$
3. Run the first raw Kaliski inverse on d_x with $t_1 = 404$.
4. Compute
$$\ell = d_y(-d_x^{-1} 2^{404}) 2^{-404} = -\lambda.$$
The multiplication here uses the default Karatsuba write-into-zero path.
5. Use the combined affine block:
$$A = \ell^2, \qquad B = A - 3x_Q,$$
$$\texttt{target\_y} \leftarrow d_y + \ell B = y_R + y_Q,$$
$$\texttt{target\_x} \leftarrow A - d_x - 3x_Q = x_R - x_Q.$$

6. Run the second raw Kaliski inverse on $x_R - x_Q$ with $t_2 = 400$.

7. Double the slope register 400 times and add

$$I_2(y_R + y_Q)$$

into it, cleaning the slope register to zero.

8. Subtract the classical y_Q:

$$y_R + y_Q \mapsto y_R.$$

9. Add the classical x_Q:

$$x_R - x_Q \mapsto x_R.$$

10. Free all non-output quantum scratch.

9 Measured No-Environment Result

A no-env release run of the submitted path produced:

Metric	Value
Emitted operations	28,677,221
Qubits	2,715
Classical bits	2,339,473
Average executed Toffoli	3,942,753.000
Average executed Clifford	22,627,103.228
Total Toffoli over 9,024 shots	35,579,403,072
Total Clifford over 9,024 shots	204,186,979,531

The same run reported:

$$\text{classical mismatches} = 0, \qquad \text{phase-garbage batches} = 0, \qquad \text{ancilla-garbage batches} = 0.$$

The submission also runs an internal alternate-seed diagnostic on the default path before the main evaluator simulation. In the measured no-env run, all five alternate seeds over 4,096 shots each had zero classical mismatches, zero phase garbage, and zero ancilla garbage.

10 Summary

The result is obtained by an affine secp256k1 point-addition circuit specialized to the Solinas prime $p = 2^{256} - 2^{32} - 977$. Its main ingredients are:

- two raw Kaliski almost-inversions, used in scaled negative form;
- a combined affine x/y rearrangement that computes $x_R - x_Q$ and $y_R + y_Q$ directly;
- reversible Litinski-style schoolbook and 1-level Karatsuba multiplications;
- sparse Solinas modular reduction using $2^{256} \equiv 2^{32} + 977$;

- measurement-based uncomputation of carries, ANDs, and OR chains;
- explicit Kaliski backward passes using stored branch history;
- aggressive scratch lifetime reduction between Kaliski forward, body, and backward phases.

The evaluator accepts the circuit because the output registers match classical secp256k1 addition on the Fiat-Shamir-derived tests, and because all measurement phases and all quantum scratch are clean at the end of the forward circuit.